Runes - a newcomer's guide

A Ruby agent harness with a fabric, an identity, a memory and a face

Runes 0.4.0 | 05 November 2026 | github.com/La-Voix-du-chat-artiste/runes

What it is, how to set it up in five minutes, and the one demo to run first. The PDF next to this file is generated from this text, so the two cannot drift.

What Runes is, in one paragraph

Runes is a **sandboxed, auditable agent-execution fabric for local-first fleets**. LLM planners decompose intents into tool calls; trusted host tools execute them inside a confined workspace; `ruby.wasm` sandboxes run untrusted tool code. Everything rides on a pluggable messaging fabric (in-process, MQTT 3.1.1, MQTT 5), every agent has an Ed25519 identity, every tool call passes a default-deny capability guard, and every prompt lifecycle is appended to a durable journal you can replay. On top of that: a Roast-compatible workflow DSL, a declarative fleet layer (`.fleet.rb` worlds + rules), a Rails observatory that watches the bus, a Spinel-compiled native kernel, and a benchmark harness for contracted small-model teams.

What you can do with it

You want to...	You use
Run an agent loop over MQTT, with guarded tools and a journal	`bin/runes-daemon`, `bin/runes-client`, `bin/runes` (TUI)
Write a multi-step pipeline as one readable `.rb` file	the workflow DSL: `bin/runes-workflow execute FILE`
Declare a world of agents + rules that reacts to events	the fleet DSL: `bin/runes-daemon --fleet world.fleet.rb`
Watch everything that happens on the bus	`runes_observer/` (Rails) + `bin/runes-replay`
Serve or call MCP tools	`bin/runes-mcp`, `Runes::MCP::Client`
Generate least-privilege broker ACLs	`bin/runes-acl`
Run untrusted code in a sandbox	the WASM pool (`ruby.wasm` + WASI guests)
Prove a small-model team works (or doesn't)	`bench/swarm/` — the 24-case LFM3 benchmark

The thesis in four lines: **conducting, not writing, is the future of code**; a DSL's *semantics* are the asset; a fleet is a *world*, not a call graph; and the alternative to a SaaS is a file.

Initial setup (about 5 minutes)

Requirements: **Ruby >= 3.3** (developed on 4.0), macOS or Linux, git.

```
git clone <this repo> && cd runes
bundle install
cp config/.env.example config/.env
```

You do **not** need an API key for the first demo below. For the live demos, edit `config/.env` and set one provider key - `DEEPSEEK` (preferred: `deepseek-flash`) and/or `SYNTHETIC` / `CEREBRAS_API_KEY`. Keys stay in `config/.env`, which is gitignored; `config/.env.example` is the tracked template.

First sanity check - the offline smoke test (embedded broker, stub planner, no key, no network):

```
bundle exec ruby demo/smoke.rb
```

It boots an embedded MQTT broker, sends one prompt through the dispatcher, runs two guarded tool calls (`write_file`, `read_file`) and prints the correlated reply plus the journal entry. If that prints `smoke ok`, the whole pipeline works on your machine.

Run the test suite (fully offline, no key):

```
bundle exec rake test
```

The demo to run first: the scripted kanban (no key needed)

`examples/prospect_pipeline.rb` is a whole CRM/product pipeline in one readable file (~460 lines of DSL): it reads an **idea**, writes a `goal.md`, plans a **mission as a Mermaid kanban** (`.mmd`), executes every todo and has a second model call **verify** each one, moves the cards (Done / Blocked *with the reason preserved*), computes the next action per contact, drafts outreach (never sends it), and writes a weekly report. Every artifact is a file another tool can read.

You can run it **for real** with a provider key:

```
PROSPECT_ROOT=/tmp/my-pipeline bin/runes-workflow execute examples/prospect_pipeline.rb
```

But the demo built for newcomers runs it **scripted** - the provider seam answers each prompt deterministically, so the run is offline, instant and reproducible, while the workflow, the files and the kanban are the real thing:

```
RUNES_DEMO_SCRIPTED=1 bundle exec ruby -Ilib scripts/demo_pipeline_run.rb
```

What you should see (abridged):

```
workflow: examples/prospect_pipeline.rb
root:     .../tmp/prospect-demo
epic:     E-001  mission: former_3_partenaires_int_grateurs.mmd
todos:    3 (pass, fail, pass)
drafts:   2  references: E-001, M-001
report:   .../tmp/prospect-demo/crm/weekly-report.md

kanban:   .../tmp/prospect-demo/epics/trouver_10_pme_de_2026-11-
05/missions/former_3_partenaires_int_grateurs.mmd
---
%% Code: M-001
%% Mission: Former 3 partenaires intégrateurs
%% Statut: Blocked

kanban
Done:
- [x] Lister 50 PME cibles - pass: fait, vérifié à la main (Assigné: Jean Dupont)
- [x] Écrire la page de lancement - pass: fait, vérifié à la main (Assigné: Samir Haddad)
Blocked:
- [ ] Préparer le kit de démo - fail: le kit n'est pas encore montrable (Assigné: Marie Martin)
---
```

The artifacts it leaves behind, all under `tmp/prospect-demo/`:

file	what it is
`epics/slug>_date>/goal.md`	the idea, shaped into a goal
`epics/slug>_date>/missions/mission>.mmd `	the **Mermaid kanban** — open it in any Mermaid renderer or `$EDITOR`
`crm/people.json`	the contacts with their computed next action
`crm/outbox/person>.md`	drafted outreach (never sent)
`crm/weekly-report.md`	the weekly report, including the blocked todo's reason

Two things to notice: the kanban is a **file contract** - a human, an agent and a Rails app can all read and edit the same `.mmd`; and one todo landed in **Blocked with its verifier's reason**, not silently dropped. The suite runs this exact file end to end offline (`test/prospect_pipeline_test.rb`), so the example cannot rot.

Optional: watch the run in the observatory. Start a broker, run the demo with telemetry, then ingest and open the Rails app:

```
bundle exec ruby demo/broker.rb 1883 &        # embedded broker, port 1883
RUNES_DEMO_SCRIPTED=1 RUNES_TELEMETRY=mqtt bundle exec ruby -Ilib scripts/demo_pipeline_run.rb
bin/runes-ingest &                             # in another shell
cd runes_observer && bin/rails server         # then open /runs and /board
```

Where to go next

- `README.md` - the full tour (architecture, the seven runes, fleet layer, security model, project layout).
- `docs/DSL_POWER.md` - the whole workflow DSL on one page, the pipeline scope by scope, and the sharp edges.
- `docs/FLEET_DSL.md` - the declarative fleet layer (world + rules).
- `docs/WHY_RUNES_V2.md` - the 8-page pitch: why this is fun and why it matters.
- `STATE.md` - the system snapshot (start here in a new session); `DEVELOPMENT_LOG.md` - phase-by-phase history; `STRATEGY.md` - positioning.
- `bench/swarm/README.md` - the contracted LFM3 team benchmark (24 cases, real MLX packs; the 2026-11-04 pilot verdict is NO-GO, and the record says why).

What we're honest about

- Execution dedupe is in-process; a durable request ledger is the next step.
- Token scanning is not a sandbox - real isolation is the WASM pool.
- A2A peer cards are unauthenticated (discovery-only); signing covers envelopes, not every progress event.
- The observatory has no auth - fine on localhost.
- The LFM3 team pilot (Cycle 43) is a **negative result**, published with numbers: the same-model control beat the team, and the echo-envelope contract is the next fix.